

MAIN REPORT

Môn học: Kiểm thử phần mềm (Software Testing)
Sinh viên thực hiện: Nguyễn Hiếu Thuận
Mã số sinh viên (MSSV): 23127125
Hệ thống thử nghiệm (SUT): EShop E-commerce Platform
Công cụ tự động hóa: Playwright Test (TypeScript)
Chiến lược kiểm thử: AI-First kết hợp Human-in-the-loop Review

I. TỔNG QUAN CHIẾN LƯỢC KIỂM THỬ TỰ ĐỘNG

1.1. Mục tiêu và Phạm vi

Bài tập HW04 tập trung tự động hóa toàn diện 3 tính năng Web cốt lõi kế thừa từ bài tập HW02:

- FR-02 (Pool A):** Đăng nhập và Khóa tài khoản (Login & Account Lockout).
- FR-08 (Pool B):** Thanh toán và Giỏ hàng (Checkout & Cart Operations).
- FR-13 (Pool C):** Quản lý Thống kê & Doanh thu Admin (Admin Dashboard).

1.2. Kiến trúc giải pháp (Test Architecture)

- Mô hình Page Object Model (POM):** Tách biệt tầng giao diện UI và tầng logic kịch bản kiểm thử:
 - LoginPage.ts:** Quản lý form đăng nhập, validation HTML5, toast notification và header greeting.
 - CheckoutPage.ts:** Quản lý giỏ hàng, double-click thêm sản phẩm, điều hướng Client-side routing, can thiệp input giá tiền, áp dụng mã **SAVE10**.
 - AdminPage.ts:** Quản lý Admin portal :5174, State Machine đơn hàng (*Chờ xác nhận* → *Đã xác nhận* → *Đang giao* → *Đã giao*), xóa sản phẩm, trích xuất số liệu Dashboard qua Leaf-node DOM evaluation.
- Kiểm thử hướng dữ liệu (Data-Driven Testing):** 100% dữ liệu đầu vào được cấu trúc trong các file JSON tách biệt (**test_data/*.json**), hoàn toàn không hardcode trong file kịch bản.
- Đa dạng hóa Assertions:** Sử dụng tối thiểu 5 dạng khẳng định Playwright khác nhau:
 - expect(page).toHaveURL(...):** Kiểm tra điều hướng URL.
 - expect(locator).toBeVisible() / not.toBeVisible():** Kiểm tra hiển thị phần tử UI.
 - expect(text).toMatch(/regex/) / toBe(...):** Kiểm tra nội dung chuỗi văn bản và số tiền.
 - expect(isValid).toBe(false):** Kiểm tra ràng buộc Client-side Validation HTML5.
 - expect(isOverflowing).toBe(false):** Kiểm tra độ co giãn Responsive và tràn khung CSS (Bounding Box).

II. CHI TIẾT KỊCH BẢN KIỂM THỬ THEO TÍNH NĂNG

2.1. Feature A: FR-02 – Đăng nhập & Khóa tài khoản

- Tập dữ liệu:** **test_data/fr02_login_data.json** (12 Test Cases).
- File kịch bản:** **tests/fr02_login.spec.ts**.
- Điểm nổi bật trong thiết kế:**

- Kịch bản mô phỏng chân thực chuỗi thao tác người dùng nhập sai mật khẩu từng lần một (Lần 1 → Lần 2 → Lần 3) với độ trễ hợp lý để kích hoạt trạng thái khóa.
- Bổ sung cấu hình `test.setTimeout(45000)` cho các ca kiểm thử phạt 30 giây (TC_02 và BVA_05) giúp Playwright kiên nhẫn chờ mở khóa an toàn mà không bị ngắt timeout.
- Giữ màn hình 1.2s tại các ca kiểm tra validation rỗng để video trace ghi nhận rõ nét popup HTML5.
- **Lưu ý:** Do SUT khóa phiên đăng nhập user sau 3 lần nhập sai, các kịch bản phải được thực hiện tuần tự từ đầu đến cuối. Khi chạy các kịch bản test case cho 3 trình duyệt khác nhau, phải đảm bảo reset lại backend trước khi chạy. Điều này có thể được thực hiện bằng cách nhấn tổ hợp **Ctrl + C** để dừng backend và khởi động lại bằng `node server.js`.
- **Bảng chi tiết Test Cases thực hiện:**

ID	Tên kịch bản (Mô tả)	Kết quả mong đợi	Kết quả thực tế	Đánh giá	Ghi chú
TC_FR-02_01	Đăng nhập thành công (Happy Path)	Chuyển hướng về trang chủ (/), hiển thị lời chào Chào, Test User.	Đăng nhập thành công, chuyển hướng về trang chủ.	 PASSED	-
TC_FR-02_02	Đăng nhập thành công sau khi hết thời gian phạt (30s)	Sau 30s hết phạt, đăng nhập lại với mật khẩu đúng thành công vào trang chủ.	Hệ thống vẫn không cho phép đăng nhập sau 30s (phải tải lại trang F5).	 FAILED	Bug 1
TC_FR-02_03	Kiểm tra validation UI khi bỏ trống Email	Trình duyệt kích hoạt HTML5 validation, chặn submit form, hiện tooltip required.	Trình duyệt kích hoạt HTML5 validation, chặn submit form, hiện tooltip required.	 PASSED	-
TC_FR-02_04	Kiểm tra validation UI khi Email sai định dạng	Trình duyệt kích hoạt validation <code>type="email"</code> , chặn submit chuỗi không có @.	Ô input không kiểm tra định dạng email chuẩn, form vẫn gửi đi.	 FAILED	Bug 2
TC_FR-02_05	Kiểm tra validation UI khi bỏ trống Password	Trình duyệt kích hoạt HTML5 validation, chặn submit form, hiện tooltip required.	Trình duyệt kích hoạt HTML5 validation, chặn submit form, hiện tooltip required.	 PASSED	-
TC_FR-02_06	Kiểm tra lỗi bảo mật khi Email chưa đăng ký	Hiển thị thông báo: Đăng nhập thất bại. Vui lòng kiểm tra lại.	Hiển thị đúng thông báo lỗi theo mong đợi.	 PASSED	-
TC_FR-02_07	Kiểm tra thông báo khi sai Password lần 3 chạm ngưỡng khóa	Hiển thị thông báo tài khoản bị tạm khóa trong 30 giây.	Không thông báo tạm khóa, chỉ báo Đăng nhập thất bại. Vui lòng kiểm tra lại.	 FAILED	Bug 3

ID	Tên kịch bản (Mô tả)	Kết quả mong đợi	Kết quả thực tế	Đánh giá	Ghi chú
TC_FR-02_08	Kiểm tra từ chối đăng nhập khi tài khoản đang bị khóa (dù nhập đúng)	Từ chối đăng nhập, giữ nguyên màn hình ở trang <code>/login</code> .	Từ chối đăng nhập, giữ nguyên màn hình ở trang <code>/login</code> .	 PASSED	-
TC_FR-02_BVA_01	Kiểm tra biên dưới số lần sai ($n = 2$)	Nhập sai 2 lần → nhập đúng ở lần 3 vẫn đăng nhập thành công (chưa bị khóa).	Hệ thống khóa đăng nhập ngay từ lần nhập sai thứ 2.	 FAILED	Bug 4
TC_FR-02_BVA_02	Kiểm tra tại biên số lần sai ($n = 3$)	Nhập sai 3 lần → tài khoản bị khóa, từ chối đăng nhập dù nhập đúng mật khẩu.	Hiển thị thông báo: Đăng nhập thất bại. Vui lòng kiểm tra lại.	 PASSED	-
TC_FR-02_BVA_03	Kiểm tra vượt biên số lần sai ($n = 4$)	Nhập sai 4 lần → tài khoản vẫn bị khóa, từ chối đăng nhập dù nhập đúng mật khẩu.	Hiển thị thông báo: Đăng nhập thất bại. Vui lòng kiểm tra lại.	 PASSED	-
TC_FR-02_BVA_05	Kiểm tra tại đúng biên thời gian khóa ($t = 30s$)	Sau đúng 30 giây thời gian phạt, đăng nhập lại với mật khẩu đúng thành công.	Hệ thống vẫn từ chối đăng nhập tại đúng mốc 30s.	 FAILED	Bug 5

2.2. Feature B: FR-08 – Thanh toán & Giỏ hàng

- **Tập dữ liệu:** `test_data/fr08_checkout_data.json` (12 Test Cases).
- **File kịch bản:** `tests/fr08_checkout.spec.ts`.
- **Điểm nổi bật trong thiết kế:**
 - **100% Pure UI Flow:** Người dùng thao tác hoàn toàn qua giao diện trình duyệt, loại bỏ việc gửi request raw backend hay can thiệp DevTools máy móc.
 - **Xử lý đặc thù UI SUT:** Nút "*Thêm vào giỏ hàng*" tại trang chi tiết `/product/1` được click đúp (với khoảng nghỉ 300ms) để khắc phục lỗi không nhận lệnh ở lần bấm đầu.
 - **Điều hướng bảo toàn State:** Bắt buộc click vào nút "*Giỏ hàng*" trên Header thay vì `page.goto('/cart')` để tránh lỗi reset bộ nhớ của ứng dụng React.
 - **Kịch bản Race Condition đa vai trò (TC_06):** User thêm sản phẩm vào giỏ → Mở tab Admin :5174 xóa sản phẩm → Quay lại tab User thanh toán → Bắt lỗi Backend vẫn cho phép đặt sản phẩm đã bị xóa.
 - **Kiểm thử mã giảm giá:** Tích hợp mã `SAVE10` (giảm 10% cho đơn từ 300.000đ).
- **Lưu ý:** Do SUT giữ nguyên dữ liệu giỏ hàng của user và danh sách sản phẩm sau khi admin xóa trong suốt phiên chạy, các kịch bản phải được thực hiện tuần tự từ đầu đến cuối. Khi chạy các kịch bản test case cho 3 trình duyệt khác nhau, phải đảm bảo reset lại backend trước khi chạy. Điều này có thể được thực hiện bằng cách nhấn tổ hợp `Ctrl + C` để dừng backend và khởi động lại bằng `node server.js`.

• **Bảng chi tiết Test Cases thực hiện:**

ID	Tên kịch bản (Mô tả)	Kết quả mong đợi	Kết quả thực tế	Đánh giá	Ghi chú
TC_FR-08_01	Thanh toán đơn hàng thành công và tự động làm rỗng giỏ hàng	Đặt hàng thành công, giỏ hàng tự động làm sạch (0 sản phẩm).	Thanh toán thành công nhưng giỏ hàng không tự động xóa sản phẩm cũ.	● FAILED	Bug 6, Bug 12
TC_FR-08_02	Kiểm tra bảo mật: Sửa tổng tiền thành 0 VNĐ	Hệ thống phát hiện sai lệch giá tiền, từ chối tạo đơn hàng 0 VNĐ.	Hệ thống cho phép sửa input và tạo đơn hàng 0 VNĐ thành công.	● FAILED	Bug 7
TC_FR-08_03	Kiểm tra bảo mật: Sửa tổng tiền thành số âm (-10 VNĐ)	Hệ thống phát hiện giá âm, từ chối tạo đơn hàng.	Hệ thống chấp nhận thanh toán đơn hàng có giá trị âm.	● FAILED	Bug 8
TC_FR-08_04	Kiểm tra bảo mật: Giỏ hàng trống nhưng gửi yêu cầu Checkout	Chặn đặt hàng khi giỏ hàng rỗng, hiển thị cảnh báo không cho thanh toán.	Hệ thống cho phép tạo đơn hàng ma (Ghost Order) khi giỏ hàng trống.	● FAILED	Bug 9
TC_FR-08_05	Kiểm tra áp dụng mã giảm giá hợp lệ SAVE10 (Giảm 10%)	Áp dụng mã thành công, giảm 10% tổng tiền thanh toán.	Áp dụng mã SAVE10 thành công, thanh toán giảm giá đúng.	● PASSED	-
TC_FR-08_06	Kiểm tra Race Condition: Sản phẩm trong giỏ bị Admin xóa trong kho	Hệ thống kiểm tra kho, báo lỗi Sản phẩm không còn tồn tại , từ chối tạo đơn.	Hệ thống vẫn tạo đơn hàng thành công cho sản phẩm đã bị xóa.	● FAILED	Bug 10
TC_FR-08_09	Kiểm tra chặn thanh toán trên giao diện UI khi chưa đăng nhập	Chuyển hướng người dùng chưa đăng nhập về trang /login .	Chuyển hướng chính xác về /login .	● PASSED	-
TC_FR-08_BVA_01	Kiểm tra biên dưới số lượng sản phẩm ($N = 0$)	Chặn thêm sản phẩm và từ chối thanh toán khi số lượng = 0.	Hệ thống chấp nhận thanh toán đơn hàng có số lượng bằng 0.	● FAILED	Bug 11
TC_FR-08_BVA_02	Kiểm tra tại biên số lượng tối thiểu ($N = 1$)	Thanh toán thành công 1 sản phẩm với giá 30.000.000đ.	Thanh toán thành công.	● PASSED	-

ID	Tên kịch bản (Mô tả)	Kết quả mong đợi	Kết quả thực tế	Đánh giá	Ghi chú
TC_FR-08_BVA_03	Kiểm tra vượt biên số lượng tối thiểu ($N = 2$)	Thanh toán thành công 2 sản phẩm với giá 60.000.000đ.	Thanh toán thành công.	 PASSED	-
TC_FR-08_BVA_04	Kiểm tra sát dưới giới hạn Max Int 32-bit ($N = 71$)	Thanh toán an toàn 71 sản phẩm (2.13 tỷ VNĐ), không tràn số.	Thanh toán thành công an toàn.	 PASSED	-
TC_FR-08_BVA_05	Kiểm tra vượt biên Max Int 32-bit ($N = 72$)	Xử lý an toàn đơn hàng 72 sản phẩm (2.16 tỷ VNĐ), không gây crash backend.	Thanh toán thành công an toàn.	 PASSED	-

2.3. Feature C: FR-13 – Admin Dashboard (Thống kê & Doanh thu)

- **Tập dữ liệu:** `test_data/fr13_dashboard_data.json` (12 Test Cases).
- **File kịch bản:** `tests/fr13_dashboard.spec.ts`.
- **Điểm nổi bật trong thiết kế:**
 - **Tích lũy trạng thái Database tuần tự:** Do SUT giữ nguyên dữ liệu đơn hàng trong suốt phiên chạy, kịch bản được sắp xếp tuần tự (`TC_01` → `BVA_01` → `TC_02` → `TC_03` → `BVA_02` → `TC_04` → `TC_05` → `TC_06` → `BVA_03` → `BVA_04` → `BVA_05` → `BVA_06`).
 - **Vòng đời State Machine:** Admin duyệt đơn qua các bước chuẩn: *Chờ xác nhận* → *Đã xác nhận* → *Đang giao* → *Đã giao*.
 - **Kiểm thử biên Max Int 32-bit:** Tạo đơn hàng 1.05 tỷ (nhân đôi thành 2.10 tỷ) và đơn hàng 1.08 tỷ (nhân đôi thành 2.16 tỷ vượt mốc 2.147.483.647) để kiểm tra chống crash server.
 - **Kiểm thử Responsive & Tràn khung (UI Layout Overflow):** Sử dụng `page.setViewportSize({ width: 768, height: 800 })` và so sánh tọa độ `textRect.right > cardRect.right` để phát hiện lỗi vỡ layout khi số tiền hoặc số đơn hàng quá lớn.
- **Lưu ý:** Do SUT giữ nguyên dữ liệu đơn hàng của trang admin trong suốt phiên chạy, kịch bản phải được thực hiện tuần tự từ đầu đến cuối. Khi chạy các kịch bản test case cho 3 trình duyệt khác nhau, phải đảm bảo reset lại backend trước khi chạy. Điều này có thể được thực hiện bằng cách nhấn tổ hợp `Ctrl + C` để dừng backend và khởi động lại bằng `node server.js`.
- **Bảng chi tiết Test Cases thực hiện:**

ID	Tên kịch bản (Mô tả)	Kết quả mong đợi	Kết quả thực tế	Đánh giá	Ghi chú
TC_FR-13_01	Dashboard khi Database trống hoàn toàn (0 đơn hàng)	Hiển thị Tổng số đơn: 0, Tổng doanh thu: 0 đ.	Hiển thị chính xác 0 đơn hàng và 0 đ.	 PASSED	-
TC_FR-13_BVA_01	Kiểm tra biên dưới số lượng đơn ($N = 0$)	Đếm chính xác số lượng đơn hàng là 0.	Đếm chính xác 0 đơn hàng.	 PASSED	-

ID	Tên kịch bản (Mô tả)	Kết quả mong đợi	Kết quả thực tế	Đánh giá	Ghi chú
TC_FR-13_02	Đếm số đơn nhưng không tính doanh thu khi chưa giao (1 đơn Pending)	Tổng số đơn: 1, Tổng doanh thu giữ nguyên 0 đ.	Đếm 1 đơn hàng và doanh thu hiển thị 0 đ.	 PASSED	-
TC_FR-13_03	Tính doanh thu khi đơn hàng chuyển sang Đã giao (Delivered 30M)	Tổng doanh thu hiển thị đúng 30.000.000 đ.	Doanh thu hiển thị thành 60.000.000 đ (bị nhân đôi x2).	 FAILED	Bug 13
TC_FR-13_BVA_02	Sát biên dưới số lượng đơn ($N = 1$)	Đếm chính xác số lượng đơn hàng là 1.	Đếm chính xác 1 đơn hàng.	 PASSED	-
TC_FR-13_04	Cập nhật số lượng khi thêm đơn mới và giữ nguyên doanh thu (1 Delivered + 1 Pending)	Tổng số đơn tăng lên 2, Tổng doanh thu giữ nguyên 60.000.000 đ.	Hiển thị đúng 2 đơn hàng và 60.000.000 đ.	 PASSED	-
TC_FR-13_05	Tính doanh thu chứa đơn hàng 0 VNĐ (Delivered)	Tổng số đơn tăng lên 3, Tổng doanh thu giữ nguyên 60.000.000 đ.	Hiển thị đúng 3 đơn hàng và 60.000.000 đ.	 PASSED	-
TC_FR-13_06	Tính doanh thu chứa đơn hàng giá trị âm (Delivered -30M triệt tiêu thành 0đ)	Tổng số đơn tăng lên 4, Tổng doanh thu triệt tiêu về 0 đ.	Hiển thị đúng 4 đơn hàng và 0 đ.	 PASSED	-
TC_FR-13_BVA_03	Biên cận dưới giới hạn Max Int 32-bit (1.05 tỷ VNĐ)	Hệ thống tính toán và hiển thị an toàn, không tràn số.	Hiển thị số liệu an toàn, không crash.	 PASSED	-
TC_FR-13_BVA_04	Vượt biên Max Int 32-bit (Tổng 1.08 tỷ $\rightarrow$ x2 = 2.16 tỷ VNĐ)	Hệ thống tính toán và hiển thị an toàn, không tràn số.	Hiển thị số liệu an toàn, không crash.	 PASSED	-
TC_FR-13_BVA_05	Giới hạn hiển thị UI (Vỡ Layout Doanh thu khi số tiền 300 nghìn tỷ VNĐ)	Thẻ Card responsive, chữ số không tràn ra ngoài viền khung.	Chữ số tiền tệ bị tràn ra ngoài viền khung Card (<code>textRect.right > cardRect.right</code>).	 FAILED	Bug 14

ID	Tên kịch bản (Mô tả)	Kết quả mong đợi	Kết quả thực tế	Đánh giá	Ghi chú
TC_FR-13_BVA_06	Hiển thị UI số lượng đơn hàng (N cực lớn 999.999.999.999 đơn)	Thẻ Card responsive, chữ số không tràn ra ngoài viền khung.	Chữ số đếm đơn hàng bị tràn ra ngoài viền khung Card.	● FAILED	Bug 15

III. PHÂN TÍCH AI GAP & RÀ SOÁT BỞI CON NGƯỜI (HUMAN REVIEW & GAP ANALYSIS)

Trong quá trình áp dụng chiến lược AI-First, mô hình AI đã bộc lộ nhiều điểm yếu và sai lầm nghiêm trọng xuất phát từ việc thiếu khả năng trải nghiệm trực quan trên trình duyệt thực tế. Dưới đây là phân tích chi tiết các lỗi của AI và giải pháp tôi đã thực hiện:

- 1. Lỗi Reset giỏ hàng do `page.goto`
- 2. Lỗi Selector ảo & Gộp chuỗi toàn trang
- 3. Lỗi Đánh timeout khi chờ phạt 30s
- 4. Lỗi CSS Selector không chuẩn
- 5. Một số lỗi khác

3.1. Phân tích chi tiết 4 lỗi lớn của AI và Cách con người sửa chữa:

1. Lỗi điều hướng máy móc làm xóa sạch giỏ hàng (Full Page Reload Bug)

- **Hành vi sai của AI:** Khi thực hiện kịch bản thanh toán FR-08, AI tự động dùng lệnh `page.goto('http://localhost:5173/cart')` sau khi thêm sản phẩm vào giỏ. Do SUT React chỉ lưu giỏ hàng trên memory cục bộ mà không đồng bộ LocalStorage/Database, việc gọi `page.goto()` đã kích hoạt tải lại toàn bộ trang (Full Page Reload), làm xóa sạch dữ liệu giỏ hàng và khiến hàng loạt test case bị fail sai thực tế.
- **Chỉnh sửa của con người:** Nhận diện được nguyên nhân từ luồng trải nghiệm thực tế, tôi đã cập nhật `CheckoutPage.ts` chuyển sang phương thức click vào nút "Giỏ hàng" trên thanh Header (`page.locator('header, nav').getByText(/^Giỏ hàng$/i).first().click()`). Điều này duy trì Client-side routing của React và bảo toàn trạng thái giỏ hàng xuyên suốt kịch bản.

2. Lỗi gộp chuỗi toàn trang do Selector không xác định phần tử lá

- **Hành vi sai của AI:** Trong FR-13, AI viết selector `div:has-text("₫")` để đọc số tiền trên Dashboard. Tuy nhiên, selector này đã bắt trúng thẻ `<div>` gốc của toàn bộ trang web (vì cả trang web có chứa ký tự `₫`), dẫn tới việc hàm trả về toàn bộ text của Sidebar, Header và Dashboard ("`EShop AdminDashboardDanh mục... 0 ₫Tổng số đơn hàng1`"), làm sai lệch assertion.
- **Chỉnh sửa của con người:** Tôi đã tái cấu trúc hàm `getRevenueText()` trong `AdminPage.ts` bằng JavaScript DOM evaluation, bóc tách chính xác theo từng dòng (`innerText lines`) bên trong thẻ con nhỏ nhất chứa tiêu đề "`Tổng doanh thu`", đảm bảo chỉ lấy đúng phần tử lá hiển thị số tiền.

3. Lỗi ngắt Test Timeout khi chờ hết thời gian phạt khóa tài khoản

- **Hành vi sai của AI:** AI sinh lệnh `await page.waitForTimeout(30000)` trong kịch bản `TC_FR-02_02` và `BVA_05` nhưng không tính đến việc Playwright có giới hạn timeout mặc định là 30.000ms cho mỗi test case. Khi kịch bản thực hiện 3 lần nhập sai (~2s) rồi chờ tiếp 30s → Tổng thời gian chậm 32s → Playwright tự động ngắt và báo lỗi `Test timeout of 30000ms exceeded` trước khi kịp đăng nhập lại.
- **Chỉnh sửa của con người:** Tôi đã bổ sung cấu hình `test.setTimeout(45000)` vào đầu mỗi test case chờ phạt, giúp Playwright chạy trọn vẹn kịch bản.

4. Lỗi sử dụng Pseudo-selector `:contains()` không chuẩn trong DOM Evaluation

- **Hành vi sai của AI:** Trong `BVA_05` và `BVA_06`, AI sử dụng cú pháp `querySelector('div:has(> *:contains("Tổng doanh thu"))')`. Cú pháp `:contains()` là của jQuery/Playwright engine nhưng KHÔNG hợp lệ trong hàm `document.querySelector` chuẩn của trình duyệt, gây ra lỗi vãng `SyntaxError`.
- **Chỉnh sửa của con người:** Tôi đã viết lại hàm DOM evaluation bằng các phương thức duyệt mảng chuẩn (`Array.from(document.querySelectorAll(...)).filter(...)`), kết hợp với lệnh `page.setViewportSize()` để kiểm tra Responsive co giãn màn hình thực sự.

5. Một số lỗi khác:

- **Hành vi sai của AI:** Trong quá trình thiết kế test script, AI gặp 1 số vấn đề với việc chuyển đổi từ đặc tả test case ra thành test script vì thiếu đi hành vi thực tế của giao diện. Ví dụ như việc nhận diện được chính xác vị trí button, hay cách thức vận hành của giao diện. Dẫn đến 1 số lỗi sai khi AI tạo ra test script.
- **Chỉnh sửa của con người:** Tôi đã bổ sung các mô tả hành vi thực tế, chỉ rõ vị trí của các element có thể tương tác và các bước thực hiện theo trình tự logic để AI có thể hiểu và tạo ra test script chính xác hơn.

IV. GIẢI TRÌNH CÁC TRƯỜNG HỢP KHÔNG THỂ TỰ ĐỘNG HÓA (UN-AUTOMATABLE EDGE CASES)

Trong bài tập HW02, có một số kịch bản kiểm thử giả định can thiệp sâu vào tầng hệ thống:

1. **Can thiệp F12 sửa đổi Auth Token (`TC_FR-08_07` & `TC_FR-08_08 cũ`):** Kịch bản này yêu cầu tester mở DevTools F12 trong khi đang thanh toán, trực tiếp chỉnh sửa chuỗi JWT Token trong Application Storage thành token giả hoặc token của người dùng khác để kiểm tra lỗi bảo mật IDOR.
 - *Lý do không tự động hóa theo luồng UI phổ thông:* Thao tác này mang tính chất Penetration Testing / API Security Testing can thiệp sâu vào header, không phản ánh hành vi tự nhiên của người dùng trên trình duyệt.
 - *Giải pháp thay thế chuẩn mực:* Tôi đã yêu cầu AI thay thế bằng kịch bản **Race Condition đa vai trò (`TC_FR-08_06`)** hoàn toàn tự nhiên: User thêm sản phẩm vào giỏ → Admin mở tab quản trị xóa sản phẩm đó → User bấm xác nhận thanh toán. Kịch bản này kiểm thử tính nhất quán dữ liệu phân tán (Distributed State Consistency) mà vẫn hoàn toàn tự động hóa được trên UI.